CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

UNIT-3 SOFTWARE DESIGN

SUBJECTCODE : CS4405

SUBJECT NAME : SOFTWARE ENGINEERING

DEPARTMENT : COMPUTER SCIENCE ENGINEERING

YEAR/SEM : II/IV

ACADEMIC YEAR: 2025-2026

BATCH 2024-2028

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

CS4405 SOFTWARE ENGINEERING **L T P C**
3 0 2 4

COURSE OBJECTIVES

- **To understand Software Engineering Lifecycle Models.**
- **To perform software requirements analysis.**
- **To gain knowledge of the System Analysis and Design concepts.**
- **To understand software testing and maintenance approaches.**
- **To work on project management scheduling using DevOps.**

UNIT III SOFTWARE DESIGN **9**

Software Design Process and Concepts – Design Principles: Coupling, Cohesion, Functional Independence – Design Patterns: Model-View-Controller, Publish-Subscribe, Adapter, Command, Strategy, Observer, Proxy, Façade – Architectural Styles: Layered, Client-Server, Tiered, Pipe and Filter – User Interface Design – Modern Design Practices: Microservices Architecture, RESTful Services

After Completion of the course, Students will be able:

S. No.	CO Statement	RBT
1	Understand and apply various software development lifecycle models, with a focus on agile methodologies, to effectively manage software projects and adapt to changing requirements.	K2,K3
2	Perform comprehensive software requirements analysis and develop clear specifications using formal methods and techniques, ensuring the alignment of software solutions with stakeholder needs.	K4,K6
3	Employ appropriate modelling techniques for system analysis and design, incorporating architectural styles and design patterns to architect scalable and maintainable software systems	K3,K6
4	Design and implement robust testing strategies to validate system functionality, reliability, and performance, ensuring the effective maintenance and evolution of software systems over time.	K3,K5,K6
5	Evaluate project management approaches, including DevOps principles, and develop effective project plans and schedules to optimize project outcomes and manage software development projects efficiently.	K6,K5

CO-PO Mapping 1-low, 2-medium,3-high

	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
C405.1	3	2	2	1	2	1	1	1	1	2	1	2	2	1	-
C405.2	3	3	2	2	2	1	1	1	1	2	1	2	3	2	-
C405.3	3	2	3	2	2	1	1	1	1	3	1	2	3	3	-
C405.4	3	2	3	3	2	1	1	1	1	2	1	2	3	2	-
C405.5	2	1	2	1	3	1	1	1	3	3	3	2	2	3	-
Average	2.8	2.0	2.4	1.8	2.2	1.0	1.0	1.0	1.4	2.4	1.4	2.4	1.4	2.0	

Course Code /Title:CC4405-
SOFTWARE ENGINEERING
UNIT:3

Software Design

Software Design is the process of transforming user requirements into a suitable form, which helps the programmer in software coding and implementation. During the software design phase, the design document is produced, based on the customer requirements as documented in the SRS document. Hence, this phase aims to transform the SRS document into a design document.

The following items are designed and documented during the design phase:

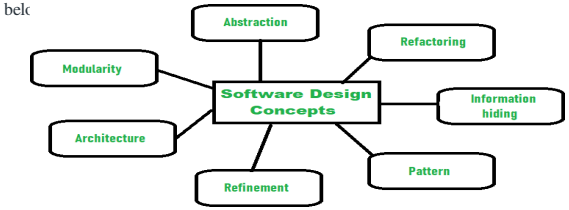
- Different modules are required.
- Control relationships among modules.
- Interface among different modules.
- Data structure among the different modules.
- Algorithms are required to be implemented among the individual modules.

Objectives of Software Design:

1. Correctness:
   A good design should be correct i.e., it should correctly implement all the functionalities of the system.
2. Efficiency:
   A good software design should address the resources, time, and cost optimization issues.
3. Flexibility:
   A good software design should have the ability to adapt and accommodate changes easily. It includes designing the software in a way, that allows for modifications, enhancements, and scalability without requiring significant rework or causing major disruptions to the existing functionality.
4. Understandability:
   A good design should be easily understandable, it should be modular, and all the modules are arranged in layers.
5. Completeness:
   The design should have all the components like data structures, modules, and external interfaces, etc.
6. Maintainability:
   A good software design aims to create a system that is easy to understand, modify, and maintain over time. This involves using modular and well-structured design principles e.g.,(employing appropriate naming conventions and providing clear documentation). Maintainability in software Design also enables developers to fix bugs, enhance features, and adapt the software to changing requirements without excessive effort or introducing new issues.

Software Design Concepts:

Concepts are defined as a principal idea or invention that comes into our mind or in thought to understand something. The **software design concept** simply means the idea or principle behind the design. It describes how you plan to solve the problem of designing software, the logic, or thinking behind how you will design software. It allows the software engineer to create the model of the system or software or product that is to be developed or built. The software design concept provides a supporting and essential structure or model for developing the right software. There are many concepts of software design and some of them are given belo



Points should be considered while Designing Software:

1. Abstraction- (Hide Irrelevant data)
   Abstraction simply means to hide the details to reduce complexity and increases efficiency or quality. Different levels of Abstraction are necessary and must be applied at each stage of the design process so that any error that is present can be removed to increase the efficiency of the software solution and to refine the software solution. The solution should be described in broad ways that cover a wide range of different things at a higher level of abstraction and a more detailed description of a solution of software should be given at the lower level of abstraction.
2. Modularity- (subdivide the system)
   Modularity simply means dividing the system or project into smaller parts to reduce the complexity of the system or project. In the same way, modularity in design means subdividing a system into smaller parts so that these parts can be created independently and then use these parts in different systems to perform different functions. It is necessary to divide the software into components known as modules because nowadays, there are different software available like Monolithic software that is hard to grasp for software engineers. So, modularity in design has now become a trend and is also important. If the system contains fewer components then it would mean the system is complex which requires a lot of effort (cost) but if we are able to divide the system into components then the cost would be small.
3. Architecture- (design a structure of something)
   Architecture simply means a technique to design a structure of something. Architecture in designing software is a concept that focuses on various elements and the data of the structure. These components interact with each other and use the data of the structure in architecture.
4. Refinement- (removes impurities)
   Refinement simply means to refine something to remove any impurities if present and increase the quality. The refinement concept of software design is actually a process of developing or presenting the software or system in a detailed manner that means to elaborate a system or software. Refinement is very necessary to find out any error if present and then to reduce it.

5. Pattern- (a Repeated form)
   The pattern simply means a repeated form or design in which the same shape is repeated several times to form a pattern. The pattern in the design process means the repetition of a solution to a common recurring problem within a
   certain context.
6. Information Hiding – Hide the Information
   Information hiding simply means to hide the information so that it cannot be accessed by an unwanted party. In software design, information hiding is
   achieved by designing the modules in a manner that the information gathered or contained in one module is hidden and can't be accessed by any other
   modules.
7. Refactoring-(Reconstruct something)
   Refactoring simply means reconstructing something in such a way that it does not affect the behavior of any other features. Refactoring in software design means reconstructing the design to reduce complexity and simplify it without impacting the behavior or its functions. Fowler has defined refactoring as "the process of changing a software system in a way that it won't impact the
   behavior of the design and improves the internal structure".

Different levels of Software Design:

There are three different levels of software design. They are:

1. Architectural Design:

   The architecture of a system can be viewed as the overall structure of the system & the way in which structure provides conceptual integrity of the system. The architectural design identifies the software as a system with many components interacting with each other. At this level, the designers get the idea of the proposed solution domain.

2. **Preliminary or high-level design:**

   Here the problem is decomposed into a set of modules, the control relationship among various modules identified, and also the interfaces among various modules are identified. The outcome of this stage is called the program architecture. Design representation techniques used in this stage are structure chart and UML.

3. Detailed Design:
   Once the high-level design is complete, a detailed design is undertaken. In detailed design, each module is examined carefully to design the data structure and algorithms. The stage outcome is documented in the form of a module specification document.

II. Design Process:

The design phase of software development deals with transforming the customer requirements as described in the SRS documents into a form implementable using a programming language. The software design process can be divided into the following three levels or phases of design:

1. Interface Design
2. Architectural Design
3. Detailed Design

Elements of a System

1. **Architecture:** This is the conceptual model that defines the structure, behavior, and views of a system. We can use flowcharts to represent and illustrate the architecture.
2. **Modules:** These are components that handle one specific task in a system. A combination of the modules makes up the system.
3. **Components:** This provides a particular function or group of related functions. They are made up of modules.
4. **Interfaces:** This is the shared boundary across which the components of a system exchange information and relate.
5. **Data:** This is the management of the information and data flow.

Interface Design

Interface design is the specification of the interaction between a system and its environment. This phase proceeds at a high level of abstraction with respect to the inner workings of the system i.e, during interface design, the internal of the systems are completely ignored, and the system is treated as a black box. Attention is focused on the dialogue between the target system and the users, devices, and other systems with which it interacts. The design problem statement produced during the problem analysis step should identify the people, other systems, and devices which are collectively called agents. Interface design should include the following details:

1. Precise description of events in the environment, or messages from agents to which the system must respond.
2. Precise description of the events or messages that the system must produce.
3. Specification of the data, and the formats of the data coming into and going out of the system.
4. Specification of the ordering and timing relationships between incoming events or messages, and outgoing events or outputs.

Architectural Design

Architectural design is the specification of the major components of a system, their responsibilities, properties, interfaces, and the relationships and interactions between them. In architectural design, the overall structure of the system is chosen, but the internal details of major components are ignored. Issues in architectural design includes:

1. Gross decomposition of the systems into major components.
2. Allocation of functional responsibilities to components.
3. Component Interfaces.
4. Component scaling and performance properties, resource consumption properties, reliability properties, and so forth.

Communication and interaction between components.

The architectural design adds important details ignored during the interface design. Design of the internals of the major components is ignored until the last phase of the design.

Detailed Design

Design is the specification of the internal elements of all major system components, their properties, relationships, processing, and often their algorithms and the data structures. The detailed design may include:

1. Decomposition of major system components into program units.
2. Allocation of functional responsibilities to units.
3. User interfaces.
4. Unit states and state changes.
5. Data and control interaction between units.
6. Data packaging and implementation, including issues of scope and visibility of program elements.
7. Algorithms and data structures.

III. Design process

- The main aim of design engineering is to generate a model which shows firmness, delight and commodity.
- Software design is an iterative process through which requirements are translated into the blueprint for building the software.

Software quality guidelines

- A design is generated using the recognizable architectural styles and compose a good design characteristic of components and it is implemented in evolutionary manner for testing.
- A design of the software must be modular i.e the software must be logically partitioned into elements.
- In design, the representation of data , architecture, interface and components should be distinct.
- A design must carry appropriate data structure and recognizable data patterns.
- Design components must show the independent functional characteristic.
- A design creates an interface that reduce the complexity of connections between the components.
- A design must be derived using the repeatable method.
- The notations should be use in design which can effectively communicates its meaning.

Quality attributes

- The attributes of design name as 'FURPS' are as follows:

Functionality:

It evaluates the feature set and capabilities of the program.

Usability:
It is accessed by considering the factors such as human factor, overall aesthetics, consistency and documentation.

Reliability:
It is evaluated by measuring parameters like frequency and security of failure, output result accuracy, the mean-time-to-failure (MTTF), recovery from failure and the the program predictability.

Performance:
It is measured by considering processing speed, response time, resource consumption, throughput and efficiency.

Supportability:

- It combines the ability to extend the program, adaptability, serviceability. These three term defines the maintainability.
- Testability, compatibility and configurability are the terms using which a system can be easily installed and found the problem easily.
- Supportability also consists of more attributes such as compatibility, extensibility, fault tolerance, modularity, reusability, robustness, security, portability, scalability.

Design concepts

The set of fundamental software design concepts are as follows:

1. Abstraction

- A solution is stated in large terms using the language of the problem environment at the highest-level abstraction.
- The lower level of abstraction provides a more detail description of the solution.
- A sequence of instruction that contain a specific and limited function refers in a procedural abstraction.
- A collection of data that describes a data object is a data abstraction.

2. Architecture

- The complete structure of the software is known as software architecture.
- Structure provides conceptual integrity for a system in a number of ways.
- The architecture is the structure of program modules where they interact with each other in a specialized way.
- The components use the structure of data.
- The aim of the software design is to obtain an architectural framework of a system.
- The more detailed design activities are conducted from the framework.

3. Patterns

A design pattern describes a design structure and that structure solves a particular design problem in a specified content.

4. Modularity

- A software is separately divided into name and addressable components. Sometime they are called as modules which integrate to satisfy the problem requirements.
- Modularity is the single attribute of a software that permits a program to be managed easily.

5.Information hiding

Modules must be specified and designed so that the information like algorithm and data presented in a module is not accessible for other modules not requiring that information.

5. Functional independence

- The functional independence is the concept of separation and related to the concept of modularity, abstraction and information hiding.
- The functional independence is accessed using two criteria i.e Cohesion and coupling.

6.Cohesion

- Cohesion is an extension of the information hiding concept.
- A cohesive module performs a single task and it requires a small interaction with the other components in other parts of the program.

7.Coupling

Coupling is an indication of interconnection between modules in a structure of software.

8. Refinement

- Refinement is a top-down design approach.
- It is a process of elaboration.
- A program is established for refining levels of procedural details.
- A hierarchy is established by decomposing a statement of function in a stepwise manner till the programming language statement are reached.

9. Refactoring

- It is a reorganization technique which simplifies the design of components without changing its function behaviour.
- Refactoring is the process of changing the software system in a way that it does not change the external behaviour of the code still improves its internal structure.

10. Design classes

- The model of software is defined as a set of design classes.
- Every class describes the elements of problem domain and that focus on features of the problem which are user visible.

IV. Coupling and Cohesion – Software Engineering

Introduction:

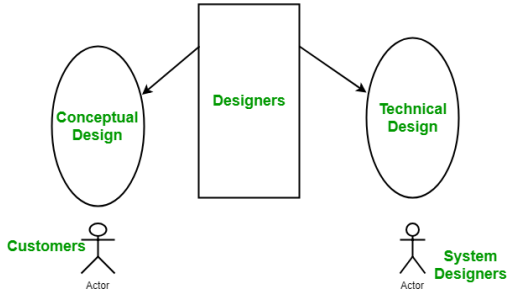
The purpose of the Design phase in the Software Development Life Cycle is to produce a solution to a problem given in the SRS (Software Requirement Specification) document. The output of the design phase is a Software Design Document (SDD).

Coupling and Cohesion are two key concepts in software engineering that are used to measure the quality of a software system's design.

Coupling refers to the degree of interdependence between software modules. High coupling means that modules are closely connected and changes in one module may affect other modules. Low coupling means that modules are independent and changes in one module have little impact on other modules.

Cohesion refers to the degree to which elements within a module work together to fulfill a single, well-defined purpose. High cohesion means that elements are closely related and focused on a single purpose, while low cohesion means that elements are loosely related and serve multiple purposes.

Both coupling and cohesion are important factors in determining the maintainability, scalability, and reliability of a software system. High coupling and low cohesion can make a system difficult to change and test, while low coupling and high cohesion make a system easier to maintain and improve. Basically, design is a two-part iterative process. The first part is Conceptual Design which tells the customer what the system will do. Second is Technical Design which allows the system builders to understand the actual hardware and software needed to solve a customer's problem.



Conceptual design of the system:

- Written in simple language i.e. customer understandable language.
- Detailed explanation about system characteristics.
- Describes the functionality of the system.
- It is independent of implementation.
- Linked with requirement document.

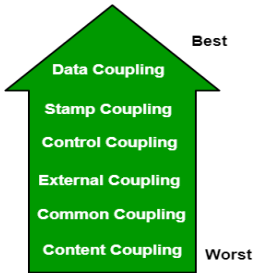
Technical Design of the System:

- Hardware component and design.
- Functionality and hierarchy of software components.
- Software architecture
- Network architecture
- Data structure and flow of data.
- I/O component of the system.
- Shows interface.

Modularization: Modularization is the process of dividing a software system into multiple independent modules where each module works independently. There are many advantages of Modularization in software engineering. Some of these are given below:

- Easy to understand the system.
- System maintenance is easy.
- A module can be used many times as their requirements. No need to write it again and again.

Coupling: Coupling is the measure of the degree of interdependence between the modules. A good software will have low coupling.

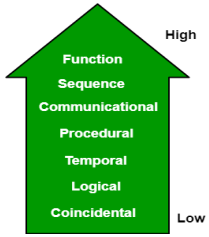


Types of Coupling:

- **Data Coupling:** If the dependency between the modules is based on the fact that they communicate by passing only data, then the modules are said to be data coupled. In data coupling, the components are independent of each other and communicate through data. Module communications don't contain tramp data. Example-customer billing system.
- **Stamp Coupling** In stamp coupling, the complete data structure is passed from one module to another module. Therefore, it involves tramp data. It may be necessary due to efficiency factors- this choice was made by the insightful designer, not a lazy programmer.
- **Control Coupling:** If the modules communicate by passing control information, then they are said to be control coupled. It can be bad if parameters indicate completely different behavior and good if parameters allow factoring and reuse of functionality. Example- sort function that takes comparison function as an argument.
- **External Coupling:** In external coupling, the modules depend on other modules, external to the software being developed or to a particular type of hardware. Ex- protocol, external file, device format, etc.
- **Common Coupling:** The modules have shared data such as global data structures. The changes in global data mean tracing back to all modules which access that data to evaluate the effect of the change. So it has got disadvantages like difficulty in reusing modules, reduced ability to control data accesses, and reduced maintainability.
- **Content Coupling:** In a content coupling, one module can modify the data of another module, or control flow is passed from one module to the other module. This is the worst form of coupling and should be avoided.

  Temporal Coupling: Temporal coupling occurs when two modules depend on the timing or order of events, such as one module needing to execute before another. This type of coupling can result in design issues and difficulties in testing and maintenance.

- **Sequential Coupling:** Sequential coupling occurs when the output of one module is used as the input of another module, creating a chain or sequence of dependencies. This type of coupling can be difficult to maintain and modify.
- **Communicational Coupling:** Communicational coupling occurs when two or more modules share a common communication mechanism, such as a shared message queue or database. This type of coupling can lead to performance issues and difficulty in debugging.
- **Functional Coupling:** Functional coupling occurs when two modules depend on each other's functionality, such as one module calling a function from another module. This type of coupling can result in tightly-coupled code that is difficult to modify and maintain.
- **Data-Structured Coupling:** Data-structured coupling occurs when two or more modules share a common data structure, such as a database table or data file. This type of coupling can lead to difficulty in maintaining the integrity of the data structure and can result in performance issues.
- **Interaction Coupling:** Interaction coupling occurs due to the methods of a class invoking methods of other classes. Like with functions, the worst form of coupling here is if methods directly access internal parts of other methods. Coupling is lowest if methods communicate directly through parameters.
- **Component Coupling:** Component coupling refers to the interaction between two classes where a class has variables of the other class. Three clear situations exist as to how this can happen. A class C can be component coupled with another class C1, if C has an instance variable of type C1, or C has a method whose parameter is of type C1,or if C has a method which has a local variable of type C1. It should be clear that whenever there is component coupling, there is likely to be interaction coupling.
- **Cohesion:** Cohesion is a measure of the degree to which the elements of the module are functionally related. It is the degree to which all elements directed towards performing a single task are contained in the component. Basically, cohesion is the internal glue that keeps the module together. A good software design will have high cohesion.



Types of Cohesion:

- **Functional Cohesion:** Every essential element for a single computation is contained in the component. A functional cohesion performs the task and functions. It is an ideal situation.
- **Sequential Cohesion:** An element outputs some data that becomes the input for other element, i.e., data flow between the parts. It occurs naturally in functional programming languages.
- **Communicational Cohesion:** Two elements operate on the same input data or contribute towards the same output data. Example- update record in the database and send it to the printer.

Procedural Cohesion: Elements of procedural cohesion ensure the order of execution. Actions are still weakly connected and unlikely to be reusable. Ex- calculate student GPA, print student record, calculate cumulative GPA, print cumulative GPA.

Temporal Cohesion: The elements are related by their timing involved. A module connected with temporal cohesion all the tasks must be executed in the same time span. This cohesion contains the code for initializing all the parts of the system. Lots of different activities occur, all at unit time.

Logical Cohesion: The elements are logically related and not functionally. Ex- A component reads inputs from tape, disk, and network. All the code for these functions is in the same component. Operations are related, but the functions are significantly different.

Coincidental Cohesion: The elements are not related(unrelated). The elements have no conceptual relationship other than location in source code. It is accidental and the worst form of cohesion. Ex- print next line and reverse the characters of a string in a single component.

Procedural Cohesion: This type of cohesion occurs when elements or tasks are grouped together in a module based on their sequence of execution, such as a module that performs a set of related procedures in a specific order. Procedural cohesion can be found in structured programming languages.

Communicational Cohesion: Communicational cohesion occurs when elements or tasks are grouped together in a module based on their interactions with each other, such as a module that handles all interactions with a specific external system or module. This type of cohesion can be found in object-oriented programming languages.

Temporal Cohesion: Temporal cohesion occurs when elements or tasks are grouped together in a module based on their timing or frequency of execution, such as a module that handles all periodic or scheduled tasks in a system. Temporal cohesion is commonly used in real-time and embedded systems.

Informational Cohesion: Informational cohesion occurs when elements or tasks are grouped together in a module based on their relationship to a specific data structure or object, such as a module that operates on a specific data type or object. Informational cohesion is commonly used in object-oriented programming.

Functional Cohesion: This type of cohesion occurs when all elements or tasks in a module contribute to a single well-defined function or purpose, and there is little or no coupling between the elements. Functional cohesion is considered the most desirable type of cohesion as it leads to more maintainable and reusable code.

Layer Cohesion: Layer cohesion occurs when elements or tasks in a module are grouped together based on their level of abstraction or responsibility, such as a module that handles only low-level hardware interactions or a module that handles only high-level business logic. Layer cohesion is commonly used in large-scale software systems to organize code into manageable layers.

Advantages of low coupling:

- Improved maintainability: Low coupling reduces the impact of changes in one module on other modules, making it easier to modify or replace individual components without affecting the entire system.
- Enhanced modularity: Low coupling allows modules to be developed and tested in isolation, improving the modularity and reusability of code.
- Better scalability: Low coupling facilitates the addition of new modules and the removal of existing ones, making it easier to scale the system as needed.

Advantages of high cohesion:

- Improved readability and understandability: High cohesion results in clear, focused modules with a single, well-defined purpose, making it easier for developers to understand the code and make changes.
- Better error isolation: High cohesion reduces the likelihood that a change in one part of a module will affect other parts, making it easier to

- Improved reliability: High cohesion leads to modules that are less prone to errors and that function more consistently,
- leading to an overall improvement in the reliability of the system.

Disadvantages of high coupling:

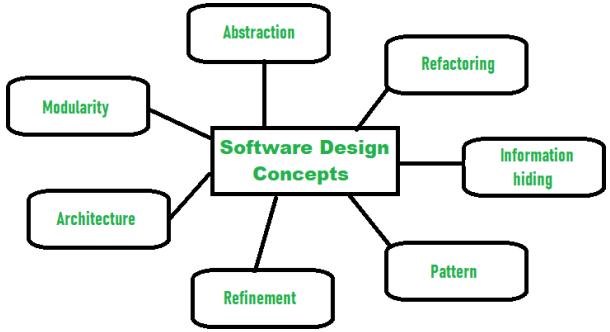
- Increased complexity: High coupling increases the interdependence between modules, making the system more complex and difficult to understand.
- Reduced flexibility: High coupling makes it more difficult to modify or replace individual components without affecting the entire system.
- Decreased modularity: High coupling makes it more difficult to develop and test modules in isolation, reducing the modularity and reusability of code.

Disadvantages of low cohesion:

- Increased code duplication: Low cohesion can lead to the duplication of code, as elements that belong together are split into separate modules.
- Reduced functionality: Low cohesion can result in modules that lack a clear purpose and contain elements that don't belong together, reducing their functionality and making them harder to maintain.
- Difficulty in understanding the module: Low cohesion can make it harder for developers to understand the purpose and behavior of a module, leading to errors and a lack of clarity.

V. Functional independence

Functional independence in software engineering refers to the principle of designing software components or modules that are self-contained and perform specific, well-defined functions without relying heavily on other parts of the system. It is a key aspect of modular design and promotes reusability, maintainability, and scalability in software development. Here are some key aspects of functional independence:



Characteristics of Functional Independence:

1. **Self-Containment**: Each software component should encapsulate its functionality and data, minimizing external dependencies.
2. **Well-Defined Interfaces**: Components should interact with each other through clearly defined interfaces, allowing them to communicate and exchange information without needing to know the internal details of one another.
3. **Loose Coupling**: Components should be loosely coupled, meaning changes to one component should have minimal impact on other components. This reduces the risk of unintended consequences and makes the system easier to maintain and modify.

4. **High Cohesion**: Components should have high cohesion, meaning that the elements within a component are closely related and work together to achieve a specific purpose. This improves the clarity and maintainability of the codebase.
5. **Encapsulation**: Components should encapsulate their internal state and behavior, exposing only the necessary functionality through well-defined interfaces. This protects the internal implementation from external interference and promotes information hiding.

Benefits of Functional Independence:

1. **Reusability**: Independent components can be reused in different parts of the system or in other projects, reducing development time and effort.
2. **Maintainability**: Changes to one component can be made without affecting other parts of the system, making maintenance and updates easier and less error-prone.
3. **Scalability**: Independent components can be scaled horizontally or vertically to accommodate changes in workload or requirements, without impacting the entire system.
4. **Testing**: Components can be tested in isolation, allowing for easier unit testing and verification of functionality.
5. **Parallel Development**: Teams can work on different components concurrently, speeding up the development process and promoting collaboration.

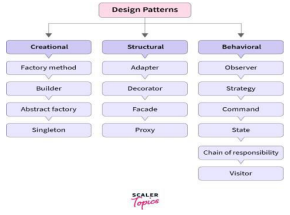
Design Practices for Achieving Functional Independence:

1. **Modular Design**: Decompose the system into smaller, manageable modules that represent cohesive units of functionality.
2. **Clear Interface Definitions**: Define clear and consistent interfaces between components, specifying the inputs, outputs, and expected behavior.
3. **Dependency Injection**: Use dependency injection to decouple components and manage dependencies dynamically.
4. **Single Responsibility Principle (SRP)**: Design components to have a single responsibility or purpose, promoting functional independence and clarity of design.

By designing software with functional independence in mind, developers can create systems that are flexible, maintainable, and scalable, capable of meeting evolving requirements and adapting to changing environments.

VI. Design Patterns

Design patterns in software engineering are reusable solutions toe common problems encountered during software design and development. They represent proven best practices distilled from the collective experience of software engineers and designers. Design patterns provide a common vocabulary for communicating design decisions and promote code reuse, maintainability, and scalability. There are several categories of design patterns, each addressing specific types of problems in software development. Some of the most common design pattern categories include:



Creational patterns focus on object creation mechanisms, providing ways to create objects while hiding the creation logic. Examples include:

1. **Singleton Pattern**: Ensures that a class has only one instance and provides a global point of access to that instance.
2. **Factory Method Pattern**: Defines an interface for creating objects, but allows subclasses to alter the type of objects that will be created.
3. **Abstract Factory Pattern**: Provides an interface for creating families of related or dependent objects without specifying their concrete classes.

Structural Patterns:

Structural patterns deal with object composition and class relationships, helping to define how classes and objects are connected to form larger structures. Examples include:

1. **Adapter Pattern**: Allows objects with incompatible interfaces to work together by providing a wrapper that converts the interface of one class into another interface that a client expects.
2. **Decorator Pattern**: Attaches additional responsibilities to an object dynamically, providing a flexible alternative to subclassing for extending functionality.
3. **Composite Pattern**: Composes objects into tree structures to represent part-whole hierarchies, allowing clients to treat individual objects and compositions of objects uniformly.

Behavioral Patterns:

Behavioral patterns focus on communication and interaction between objects, defining how objects collaborate to achieve complex behaviors. Examples include:

1. **Observer Pattern**: Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.
2. **Strategy Pattern**: Defines a family of algorithms, encapsulates each one, and makes them interchangeable, allowing clients to choose algorithms dynamically at runtime.
3. **Command Pattern**: Encapsulates a request as an object, thereby allowing for parameterization of clients with queues, requests, and operations.

Architectural Patterns:

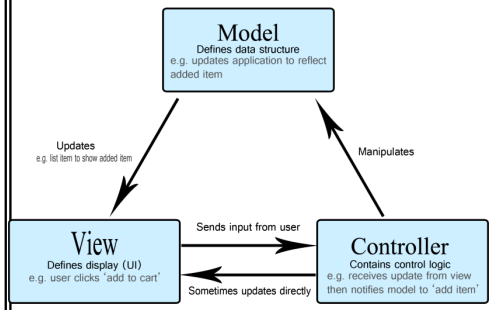
Architectural patterns provide high-level guidelines and best practices for organizing the structure and behavior of entire software systems. Examples include:

1. **Model-View-Controller (MVC)**: Separates an application into three main components - Model (data), View (presentation layer), and Controller (business logic), promoting separation of concerns and modularity.
2. **Layered Architecture**: Organizes software into layers, such as presentation, business logic, and data access, to promote scalability, maintainability, and reusability.
3. **Microservices Architecture**: Decomposes a large system into smaller, independently deployable services, each focused on a specific business capability, promoting agility, scalability, and fault isolation.

By understanding and applying design patterns appropriately, software engineers can create flexible, modular, and maintainable software systems that are robust and adaptable to change. However, it's essential to use design patterns judiciously and consider the specific requirements and constraints of each software project.

VII. Model-View-Controller (MVC)

In software engineering, the Model-View-Controller (MVC) is a design pattern that separates an application into three interconnected components: the Model, the View, and the Controller. This separation allows for a more modular, maintainable, and scalable architecture



1. Model:

The Model represents the application's data and business logic. It encapsulates the data and provides methods to manipulate and access that data. The Model responds to requests from the Controller to update its state and notifies the View of any changes. It is independent of both the View and the Controller and thus can be reused across different parts of the application.

2. View:

The View is responsible for presenting the Model's data to the user. It displays the user interface elements and interacts with the user to gather input. The View observes changes in the Model and updates itself accordingly. In traditional MVC, the View is passive and does not directly interact with the Model; it receives updates from the Controller or the Model itself.

3. Controller:

The Controller acts as an intermediary between the Model and the View. It receives input from the user via the View, processes it, and updates the Model accordingly. The Controller translates user actions into operations on the Model and updates the View based on changes in the Model. It orchestrates the flow of data and logic between the Model and the View.

Benefits of Model-View-Controller (MVC):

1. **Separation of Concerns**: MVC separates the application's concerns into distinct components, making it easier to manage and maintain.
2. **Modularity and Reusability**: Each component in MVC can be developed, tested, and modified independently, promoting code reuse and modularity.
3. **Scalability**: MVC facilitates the scalability of applications by allowing for the addition or modification of components without affecting the entire system.
4. **Support for Multiple Views**: Since the View is decoupled from the Model and Controller, it is possible to have multiple views of the same data without changing the underlying logic.
5. **Enhanced Testability**: MVC improves the testability of applications by enabling unit testing of individual components, such as the Model and Controller, in isolation.

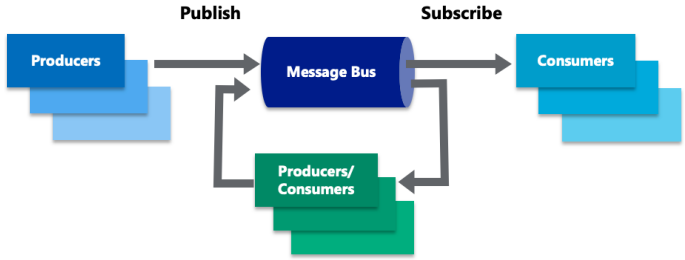
Variants of MVC:

While the traditional MVC pattern provides a clear separation of concerns, there are several variations and adaptations of the pattern, such as Model-View-Presenter (MVP), Model-View-ViewModel (MVVM), and others. These variants aim to address specific requirements or constraints of different application architectures and frameworks while preserving the core principles of separation of concerns and modularity.

In summary, the Model-View-Controller pattern is a widely used architectural pattern in software engineering for designing user interfaces and applications. It provides a structured approach to building software systems that are modular, maintainable, and scalable.

VIII. Publish Subscribe

Publish-Subscribe is a messaging pattern used in software engineering to facilitate communication between components or modules in a system. It allows for loosely coupled communication, where publishers (senders) of messages do not need to know the details of the subscribers (receivers), and vice versa. This pattern is widely used in distributed systems, event-driven architectures, and messaging systems. Here's how it works:



Key Components:

1. **Publisher**: The entity that generates and sends messages or events. It does not have any knowledge of the subscribers or how they handle the messages it publishes.
2. **Subscriber**: The entity that receives and processes messages or events published by one or more publishers. Subscribers express interest in specific types of messages or events they want to receive.
3. **Message/Event**: The data or event notification that is published by the publisher and received by the subscriber. It contains relevant information that subscribers may act upon.

How it Works:

1. **Publishing**: The publisher generates a message or event and sends it to a messaging system or a message broker without knowing which subscribers, if any, will receive it. The message broker is responsible for routing the message to the appropriate subscribers.
2. **Subscription**: Subscribers express interest in specific types of messages or events they want to receive. They register with the messaging system or message broker, indicating the types of messages they are interested in.
3. **Routing**: The messaging system or message broker receives messages from publishers and routes them to the appropriate subscribers based on their subscriptions. It ensures that messages are delivered only to the subscribers interested in receiving them.
4. **Delivery**: Subscribers receive the messages or events they are subscribed to and process them accordingly. Each subscriber can define its own logic for handling messages.

Benefits of Publish-Subscribe Pattern:

1. **Loose Coupling**: Publishers and subscribers are decoupled from each other, allowing them to evolve independently without affecting each other.
2. **Scalability**: Publish-Subscribe can scale to accommodate a large number of publishers and subscribers, as the messaging system handles the routing and delivery of messages efficiently.
3. **Flexibility**: Subscribers can dynamically subscribe and unsubscribe to different types of messages or events without impacting the publisher or other subscribers.
4. **Asynchronous Communication**: Publishers and subscribers can operate asynchronously, allowing for non-blocking communication and improved system responsiveness.
5. **Event-Driven Architecture**: Publish-Subscribe is a key pattern in event-driven architectures, where components react to events or changes in the system state.

Use Cases:

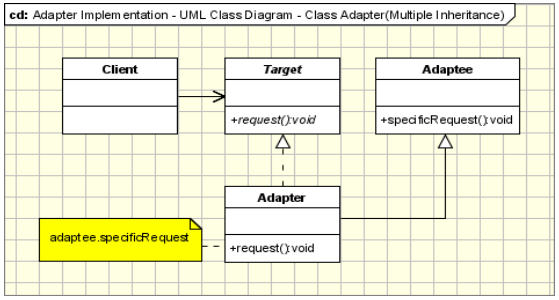
1. **Distributed Systems**: Publish-Subscribe is used to enable communication between distributed components or microservices in a system.
2. **Message Queues and Brokers**: Messaging systems and message brokers like RabbitMQ, Apache Kafka, and Amazon SNS/SQS implement the Publish-Subscribe pattern to facilitate communication between producers and consumers.
3. **Real-Time Systems**: Publish-Subscribe is used in real-time systems, such as chat applications and IoT platforms, to deliver messages and events in near real-time to interested subscribers.

In summary, the Publish-Subscribe pattern provides a flexible and scalable mechanism for communication between components in a software system, enabling loosely coupled, asynchronous, and event-driven architectures.

IX. Adapter and Command patterns

The Adapter and Command patterns are two distinct design patterns used in software engineering to solve different types of problems:

Adapter Pattern:

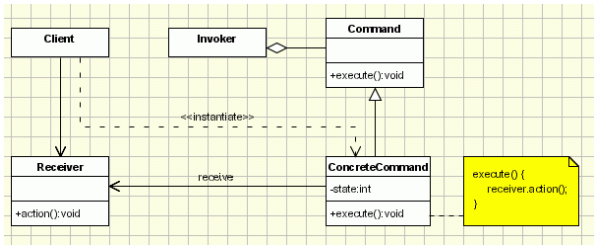


The Adapter pattern is a structural design pattern that allows objects with incompatible interfaces to work together. It acts as a bridge between two incompatible interfaces, converting the interface of one class into another interface that a client expects. The Adapter pattern typically involves creating a wrapper class that implements the expected interface and delegates the actual implementation to the adaptee.

- **Problem**: You have two classes or components with incompatible interfaces, and you want them to work together without modifying their existing code.
- **Solution**: Create an adapter class that implements the expected interface and wraps the adaptee, translating calls from the expected interface to the adaptee's interface.

Example: Suppose you have an existing class that provides functionality through a specific interface, and you want to use this functionality with another class that expects a different interface. You can create an adapter class that adapts the interface of the existing class to match the interface expected by the second class.

Command Pattern:



The Command pattern is a behavioral design pattern that encapsulates a request as an object, allowing for parameterization of clients with queues, requests, and operations. It decouples the sender of a request from its receiver, allowing for parameterization of clients with queues, requests, and operations.

- **Problem**: You want to decouple the sender of a request from its receiver, allowing for parameterization and the ability to queue requests.
- **Solution**: Encapsulate a request as an object, which includes all the necessary information to perform the request, and use a command interface to represent various types of requests.

Example: In a text editor application, you can implement the Command pattern by creating command objects for actions such as "copy," "cut," and "paste." Each command object encapsulates the operation to be performed and the receiver of the operation (e.g., the text editor). Clients can then invoke commands without knowing the details of how they are executed.

Relationship:

While both patterns are used to solve different types of problems, there might be scenarios where they can be used together. For example, you might use the Adapter pattern to adapt the interface of a third-party library to match the interface expected by a command object in the Command pattern.

In summary, the Adapter pattern is used to make two incompatible interfaces work together, while the Command pattern is used to encapsulate a request as an object, allowing for parameterization and decoupling of the sender from the receiver.

X. System Design Strategy

A good system design is to organize the program modules in such a way that are easy to develop and change. Structured design techniques help developers to deal with the size and complexity of programs. Analysts create instructions for the developers about how code should be written and how pieces of code should fit together to form a program.

Software Engineering is the process of designing, building, testing, and maintaining software. The goal of software engineering is to create software that is reliable, efficient, and easy to maintain. System design is a critical component of software engineering and involves making decisions about the architecture, components, modules, interfaces, and data for a software system.

System Design Strategy refers to the approach that is taken to design a software system. There are several strategies that can be used to design software systems, including the following:

1. Top-Down Design: This strategy starts with a high-level view of the system and gradually breaks it down into smaller, more manageable components.
2. Bottom-Up Design: This strategy starts with individual components and builds the system up, piece by piece.
3. Iterative Design: This strategy involves designing and implementing the system in stages, with each stage building on the results of the previous stage.
4. Incremental Design: This strategy involves designing and implementing a small part of the system at a time, adding more functionality with each iteration.
5. Agile Design: This strategy involves a flexible, iterative approach to design, where requirements and design evolve through collaboration between self- organizing and cross-functional teams.

The design of a system is essentially a blueprint or a plan for a solution for the system. The design process for software systems often has two levels. At the first level the focus is on deciding which modules are needed for the system, the specifications of these modules and how the modules should be interconnected. The design of a system is correct if a system built precisely according to the design satisfies the requirements of that system. The goal of the design process is not simply to produce a design for the system. Instead, the goal is to find the best possible design within the limitations imposed by the requirements and the physical and social environment in which the system will operate.

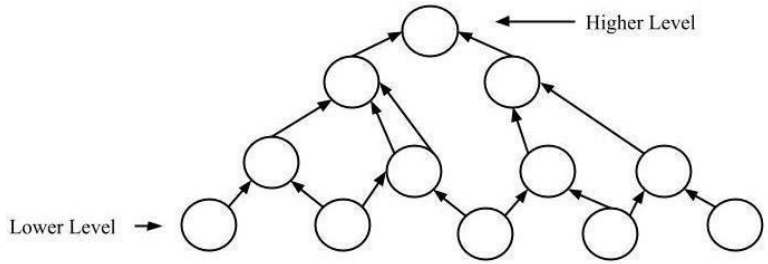
The choice of system design strategy will depend on the particular requirements of the software system, the size and complexity of the system, and the development methodology being used. A well-designed system can simplify the development process, improve the quality of the software, and make the software easier to maintain.

Importance of System Design Strategy:

1. If any pre-existing code needs to be understood, organized, and pieced together.
2. It is common for the project team to have to write some code and produce original programs that support the application logic of the system.

There are many strategies or techniques for performing system design. They are:
Bottom-up approach: The design starts with the lowest level components and subsystems. By using these components, the next immediate higher-level components and subsystems are created or composed. The process is continued till all the components and subsystems are composed into a single component, which is considered as the complete system. The amount of abstraction as the design moves to more high levels. By using the basic information existing system, when a new system needs to be created, the bottom-up strategy suits the purpose.

By using the basic information existing system, when a new system needs to be created, the bottom-up strategy suits the purpose.



Bottom-up approach

Advantages of Bottom-up approach:

- The economics can result when general solutions can be reused.
- It can be used to hide the low-level details of implementation and be merged with the top-down technique.

Disadvantages of Bottom-up approach:

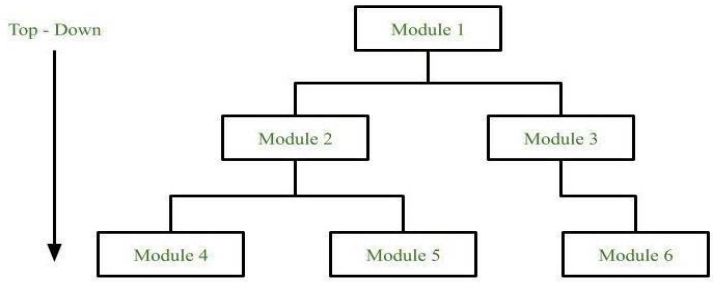
- It is not so closely related to the structure of the problem.
- High-quality bottom-up solutions are very hard to construct.
- It leads to the proliferation of 'potentially useful' functions rather than the most appropriate ones.

Top-down approach:

Each system is divided into several subsystems and components. Each of the subsystems is further divided into a set of subsystems and components. This process of division facilitates forming a system hierarchy structure. The complete software system is considered a single entity and in relation to the characteristics, the system is split into sub-systems and components. The same is done with each of the sub-systems. This process is continued until the lowest level of the system is reached. The design is started initially by defining the system as a whole and then keeps on adding definitions of the subsystems and components. When all the definitions are combined, it turns out to be a complete system.

For the solutions of the software that need to be developed from the ground level, a top- down design best suits the purpose.

Top-down approach



Top-down approach

Advantages of Top-down approach:

- The main advantage of the top-down approach is that its strong focus on requirements helps to make a design responsive according to its requirements.

Disadvantages of Top-down approach:

- Project and system boundaries tend to be application specification-oriented. Thus, it is more likely that the advantages of component reuse will be missed.
- The system is likely to miss, the benefits of a well-structured, simple architecture.
- Hybrid

  Design:

  It is a combination of both top-down and bottom-up design strategies. In this, we can reuse the modules.

Advantages of using a System Design Strategy:

1. Improved quality: A well-designed system can improve the overall quality of the software, as it provides a clear and organized structure for the software.
2. Ease of maintenance: A well-designed system can make it easier to maintain and update the software, as the design provides a clear and organized structure for the software.
3. Improved efficiency: A well-designed system can make the software more efficient, as it provides a clear and organized structure for the software that reduces the complexity of the code.
4. Better communication: A well-designed system can improve communication between stakeholders, as it provides a clear and organized structure for the software that makes it easier for stakeholders to understand and agree on the design of the software.
5. Faster development: A well-designed system can speed up the development process, as it provides a clear and organized structure for the software that makes it easier for developers to understand the requirements and implement the software.

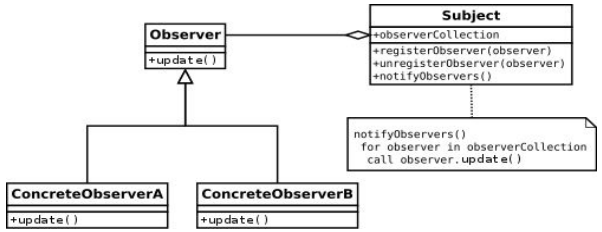
Disadvantages of using a System Design Strategy:

1. Time-consuming: Designing a system can be time-consuming, especially for large and complex systems, as it requires a significant amount of documentation and analysis.
2. Inflexibility: Once a system has been designed, it can be difficult to make changes to the design, as the process is often highly structured and documentation-intensive.

XI. The Observer, Proxy, and Facade patterns

The Observer, Proxy, and Facade patterns are three distinct design patterns used in software engineering to solve different types of problems:

Observer Pattern:

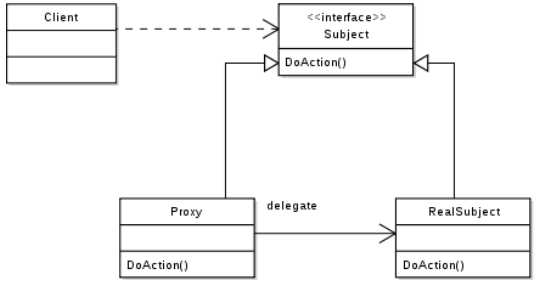


The Observer pattern is a behavioral design pattern that establishes a one-to-many dependency between objects. It defines a subscription mechanism where multiple observers (or subscribers) are notified of changes in the state of a subject (or publisher).

- **Problem**: You need to notify multiple objects when the state of another object changes without tightly coupling them together.
- **Solution**: Define a subject (or observable) class that maintains a list of its dependents (observers) and provides methods to register, unregister, and notify observers of changes in its state.

Example: The Observer pattern is commonly used in GUI frameworks, where graphical elements (observers) are notified of changes in underlying data models (subjects) and update their displays accordingly.

Proxy Pattern:

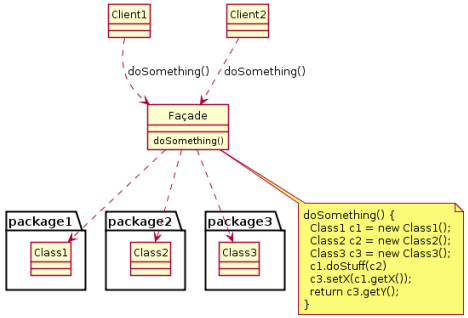


The Proxy pattern is a structural design pattern that provides a surrogate or placeholder for another object to control access to it. It acts as an intermediary or wrapper object that adds functionality to the underlying object without modifying its code.

- **Problem**: You want to add additional functionality to an existing object or control access to it without changing its interface.
- **Solution**: Create a proxy class that implements the same interface as the underlying object and forwards requests to it, optionally performing additional operations before or after the forwarding.

Example: The Proxy pattern is used in scenarios such as lazy initialization (where the actual object is created only when it is accessed for the first time), access control (where access to an object is restricted based on permissions), and logging (where access to an object is logged for auditing purposes).

Facade Pattern:



The Facade pattern is a structural design pattern that provides a unified interface to a set of interfaces in a subsystem. It defines a higher-level interface that makes it easier to use the subsystem by hiding its complexity.

- **Problem**: You want to simplify the interaction with a complex subsystem by providing a unified interface that encapsulates its functionality.
- **Solution**: Create a facade class that provides a simplified interface to the complex subsystem, hiding its implementation details and reducing its coupling with clients.

Example: The Facade pattern is commonly used in large software systems to provide simplified interfaces to subsystems such as databases, networking, or external APIs. It helps to improve the maintainability and usability of the system by encapsulating complex logic behind a simpler interface.

Relationship:
While Observer, Proxy, and Facade are separate design patterns, they can be used together in various combinations depending on the requirements of the system being designed. For example, you might use a Proxy to control access to an Observer, or you might use a Facade to simplify the interaction with a system that includes Observers and Proxies.

In summary, the Observer pattern is used to establish a dependency between objects, the Proxy pattern is used to control access to objects, and the Facade pattern is used to simplify the interaction with complex subsystems. Each pattern addresses different types of problems and can be applied independently or in combination with other patterns as needed.

XII. Architectural Design

Introduction: The software needs the architectural design to represents the design of software. IEEE defines architectural design as "the process of defining a collection of hardware and software components and their interfaces to establish the framework for the development of a computer system." The software that is built for computer-based systems can exhibit one of these many architectural styles. Each style will describe a system category that consists of

- A set of components (e.g.: a database, computational modules) that will perform a function required by the system.
- The set of connectors will help in coordination, communication, and cooperation between the components.
- Conditions that how components can be integrated to form the system.
- Semantic models that help the designer to understand the overall properties of the system.

The use of architectural styles is to establish a structure for all the components of the system

Taxonomy of Architectural styles:

1. Data centered architectures:

- A data store will reside at the center of this architecture and is accessed frequently by the other components that update, add, delete or modify the data present within the store.
- The figure illustrates a typical data centered style. The client software access a central repository. Variation of this approach are used to transform the repository into a blackboard when data related to client or data of interest for the client change the notifications to client software.
- This data-centered architecture will promote integrability. This means that the existing components can be changed and new client components can be added to the architecture without the permission or concern of other clients.
- Data can be passed among clients using blackboard mechanism.

Advantage of Data centered architecture

- Repository of data is independent of clients
- Client work independent of each other
- It may be simple to add additional clients.
- Modification can be very easy

Data centered architecture

2. Data flow architectures:
   - This kind of architecture is used when input data is transformed into output data through a series of computational manipulative components.
   - The figure represents pipe-and-filter architecture since it uses both pipe and filter and it has a set of components called filters connected by lines.
   - Pipes are used to transmitting data from one component to the next.

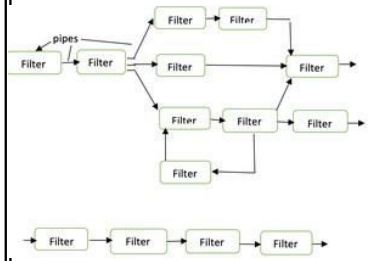
- Each filter will work independently and is designed to take data input of a certain form and produces data output to the next filter of a specified form. The filters don't require any knowledge of the working of neighboring filters.
- If the data flow degenerates into a single line of transforms, then it is termed as batch sequential. This structure accepts the batch of data and then applies a series of sequential components to transform it.

Advantages of Data Flow architecture

- It encourages upkeep, repurposing, and modification.
- With this design, concurrent execution is supported.

The disadvantage of Data Flow architecture

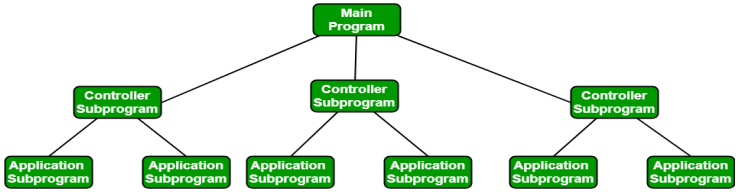
- It frequently degenerates to batch sequential system
- Data flow architecture does not allow applications that require greater user engagement.
- It is not easy to coordinate two different but related streams



Data Flow architecture

2] Call and Return architectures: It is used to create a program that is easy to scale and modify. Many sub-styles exist within this category. Two of them are explained below.

- **Remote procedure call architecture:** This components is used to present in a main program or sub program architecture distributed among multiple computers on a network.
- **Main program or Subprogram architectures:** The main program structure decomposes into number of subprograms or function into a control hierarchy. Main program contains number of subprograms that can invoke other components



4.Object Oriented architecture: The components of a system encapsulate data and the operations that must be applied to manipulate the data. The coordination and communication between the components are established via the message passing.

Characteristics of Object-Oriented architecture

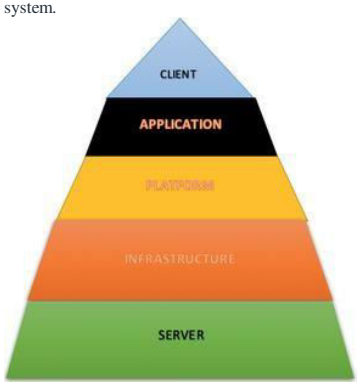
- Object protect the system's integrity.
- An object is unaware of the depiction of other items.

Advantage of Object Oriented architecture

- It enables the designer to separate a challenge into a collection of autonomous objects.
- Other objects are aware of the implementation details of the object, allowing changes to be made without having an impact on other objects.

5.Layered Architecture:

- A number of different layers are defined with each layer performing a well- defined set of operations. Each layer will do some operations that becomes closer to machine instruction set progressively.
- At the outer layer, components will receive the user interface operations and at the inner layers, components will perform the operating system interfacing(communication and coordination with OS)
- Intermediate layers to utility services and application software functions.
- One common example of this architectural style is OSI-ISO (Open Systems Interconnection-International Organisation for Standardisation) communication system.



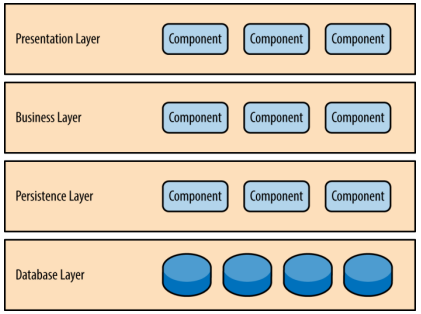
Layered architecture:

XIII Layered, Client-Server, and Tiered architectures

Layered, Client-Server, and Tiered architectures are three common architectural patterns used in software engineering to structure and organize the components of a system. Each of these patterns has its own characteristics and is suitable for different types of applications and requirements.

Layered Architecture:

Layered architecture, also known as N-tier architecture, organizes the components of a system into horizontal layers, where each layer represents a different level of abstraction and functionality. The most common layers include:

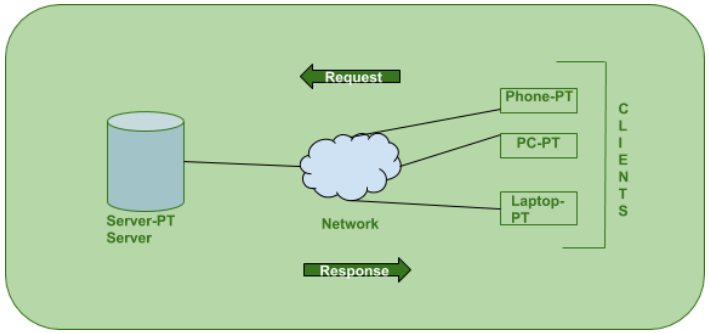


1. **Presentation Layer**: Responsible for presenting information to the user and handling user input.
2. **Business Logic Layer (or Service Layer)**: Contains the core business logic and rules of the application.
3. **Data Access Layer (or Persistence Layer)**: Provides access to data storage and manages data retrieval and manipulation.

Characteristics:

- Promotes separation of concerns and modularity.
- Each layer depends only on the layer directly beneath it.
- Changes in one layer typically do not affect other layers.
- Supports reusability and maintainability.

Client-Server Architecture:



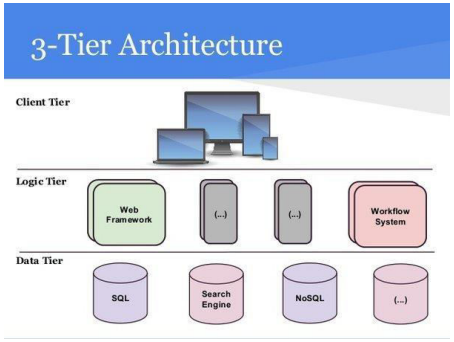
Client-server architecture is a distributed computing model where tasks or workloads are divided between clients and servers. Clients request services or resources from servers, which fulfill those requests and return responses. This architecture typically involves two main components:

1. **Client**: The user-facing part of the application that interacts with users and requests services from servers.
2. **Server**: The backend part of the application that provides services or resources to clients.

Characteristics:

- Enables scalability and flexibility by distributing tasks between clients and servers.
- Centralized management and control on the server side.
- Clients can be lightweight and platform-independent.
- Supports the development of distributed systems and web-based applications.

Tiered Architecture:



Tiered architecture is an extension of the client-server architecture that adds multiple layers of servers to handle different aspects of the application's functionality. The most common tiers include:

1. **Presentation Tier (or Client Tier)**: Handles user interaction and presentation logic.
2. **Application Tier (or Middle Tier)**: Implements business logic and application-specific functionality.
3. **Data Tier (or Backend Tier)**: Manages data storage and access.

Characteristics:

- Enhances scalability and performance by distributing tasks across multiple tiers.
- Promotes separation of concerns and modularity.
- Each tier can be scaled independently based on demand.
- Supports the development of complex and large-scale systems.

Relationship:

Layered architecture is a design approach that can be applied within both client-server and tiered architectures. In client-server and tiered architectures, the components are typically organized in layers to promote separation of concerns and modularity. However, while layered architecture focuses on functional decomposition, client-server and tiered architectures focus on distribution of tasks and resources across different components or tiers.

In summary, layered, client-server, and tiered architectures are all important architectural patterns used in software engineering to design and structure complex systems. The choice of architecture depends on factors such as scalability requirements, performance goals, and the nature of the application being developed.

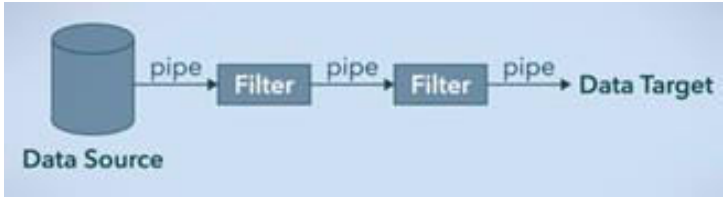
XIV. Pipe and Filter

Pipe and Filter is another architectural pattern, which has independent entities called **filters** (components) which perform transformations on data and process the input they receive, and **pipes**, which serve as connectors for the stream of data being transformed, each connected to the next component in the pipeline.

Many systems are required to transform streams of discrete data items, from input to output. Many types of transformations occur repeatedly in practice, and so it is desirable to create these as independent, reusable parts, Filters. (Len Bass, 2012)

Description of the Pattern

The pattern of interaction in the pipe-and-filter pattern is characterized by successive transformations of streams of data. As you can see in the diagram, the data flows in one direction. It starts at a data source, arrives at a filter's input port(s) where processing is done at the component, and then, is passed via its output port(s) through a pipe to the next filter, and then eventually ends at the data target.



A single filter can consume data from, or produce data to, one or more ports. They can also run concurrently and are not dependent. The output of one filter is the input of another, hence, the order is very important.

A pipe has a single source for its input and a single target for its output. It preserves the sequence of data items, and it does not alter the data passing through.

Advantages of selecting the pipe and filter architecture are as follows:

- Ensures loose and flexible coupling of components, filters.
- Loose coupling allows filters to be changed without modifications to other filters.
- Conductive to parallel processing.
- Filters can be treated as black boxes. Users of the system don't need to know the logic behind the working of each filter.
- Re-usability. Each filter can be called and used over and over again. However, there are a few

Drawbacks to this architecture and are discussed below:

- Addition of a large number of independent filters may reduce performance due to excessive computational overheads.
- Not a good choice for an interactive system.
- Pipe-and-fitter systems may not be appropriate for long-running computations.

Applications of the Pattern

In software engineering, a **pipeline** consists of a chain of processing elements (processes, threads, functions, etc.), arranged so that the output of each element is the input of the next. (Wiki, n.d.).

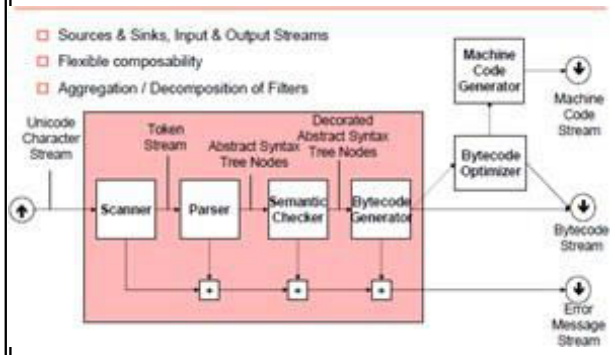
The architectural pattern is very popular and used in many systems, such as the text-based utilities in the UNIX operating system. Whenever different data sets need to be manipulated in different ways, you should consider using the pipe and filter architecture. More specific implementations are discussed below:

1. Compilers:

A compiler performs language transformation: Input is in language A and output is in language B. In order to do that the input goes through various stages inside the compiler — these stages form the pipeline. The most commonly used division consists of 3 stages: front-end, middle-end, and back-end.

The front-end is responsible for parsing the input language and performing syntax and semantic and then transforms it into an intermediate language. The middle-end takes the intermediate representation and usually performs several optimization steps on it, the resulting transformed program in is passed to the back-end which transforms it into language B.

Each level consists of several steps as well, and everything together forms the pipeline of the compiler.



Working of a compiler

2. UNIX Shell:

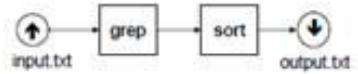
The Pipeline is one of the defining features of the UNIX shell, and obviously, the same goes for Linux, MacOS, and any other Unix-based or inspired systems.

In a nutshell, it allows you to tie the output of one program to the input of another. The benefit it brings is that you don't have to save the results of one program before you can start processing it with another. The long-term and even more important benefit is that it encourages programs to be small and simple.

There is no need for every program to include a word-counter if they can all be piped into **wc**. Similarly, no program needs to offer its own built-in pattern matching facilities, as it can be piped into **grep**.

In the provided example, the **input.txt** is read and the output is then provided to **grep** as input which searches for the pattern "text" and then passes the results to **sort**, which sorts the results and outputs into the file, **output.txt**.

Unix shell: `cat input.txt | grep "text" | sort > output.txt`



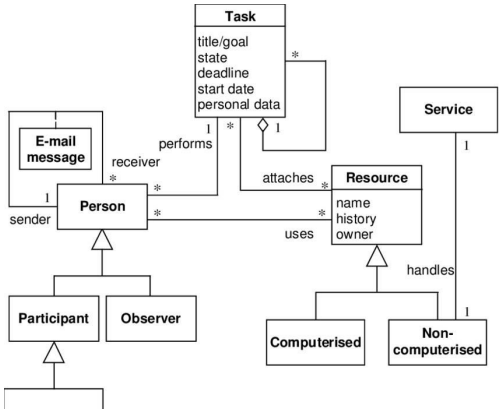
Example — Pipelining in the UNIX shell

XV. Case Study

case study that illustrates the software design process for developing a simple task management application.

Case Study: Task Management Application

A small startup company wants to develop a task management application to help its employees organize their work tasks, set deadlines, and track progress. The application needs to be accessible via web browsers and mobile devices to accommodate employees working remotely.



Requirements:

1. **User Authentication**: Users should be able to register, log in, and log out securely.
2. **Task Management**: Users should be able to create, edit, delete, and view tasks. Each task should have a title, description, deadline, priority level, and status (e.g., pending, in progress, completed).
3. **Notifications**: Users should receive notifications for approaching deadlines or updates on their tasks.
4. **Accessibility**: The application should be accessible via web browsers and mobile devices with responsive design.

5. **Security**: User data should be stored securely, and access should be restricted based on user roles and permissions.
6. **Scalability**: The application should be scalable to accommodate potential growth in the number of users and tasks.

Software Design Process:

1. **Requirements Analysis**:
   - Gather and analyze requirements from stakeholders, including end-users and management.
   - Define user stories, use cases, and acceptance criteria to capture functional and non- functional requirements.
2. **System Architecture Design**:
   - Identify the major components and modules of the system, such as user authentication, task management, notifications, and security.
   - Define the overall architecture, including client-server architecture, database schema, and APIs.
   - Choose appropriate technologies and frameworks based on requirements and constraints.
3. **Database Design**:
   - Design the database schema to store user accounts, tasks, notifications, and other relevant data.
   - Define relationships between tables, indexes, and constraints to ensure data integrity and efficiency.
4. **User Interface (UI) Design**:
   - Design the user interface for web and mobile platforms, focusing on usability, intuitiveness, and responsiveness.
   - Create wireframes, mockups, and prototypes to visualize the user interface and gather feedback from stakeholders.
5. **Implementation**:
   - Develop the application components and modules using chosen programming languages, frameworks, and libraries.
   - Implement user authentication mechanisms, task management functionalities, notifications, and security features.
   - Write unit tests, integration tests, and end-to-end tests to ensure the quality and reliability of the code.
6. **Testing and Quality Assurance**:
   - Conduct thorough testing of the application to identify and fix bugs, issues, and usability problems.
   - Perform functional testing, usability testing, performance testing, and security testing.
   - Ensure compatibility with different web browsers and mobile devices.
7. **Deployment and Maintenance**:
   - Deploy the application to production servers or cloud platforms.
   - Monitor the application's performance, security, and reliability in production.
   - Provide ongoing maintenance and support, including bug fixes, updates, and enhancements based on user feedback and evolving requirements.

Conclusion:

By following a systematic software design process, the startup company successfully develops and deploys a task management application that meets the needs of its employees. The application helps streamline workflow, improve productivity, and foster collaboration among team members, contributing to the company's success and growth.

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

NBA NIRF 101 - 150 Band Engineering 2024 A+ NAAC

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

NBA NIRF 101 - 150 Band Engineering 2024 A+ NAAC

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)